Design & Analysis of Algorithm

Unit II

Analysis of Algorithms and Complexity Theory

By Prof. Bhavana A. Khivsara

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)



WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b



Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

U2: Analysis of Algorithms and Complexity Theory

Syllabus:

- **Analysis**: Input size, best case, worst case, average case Counting Dominant operators, Growth rate, upper bounds
- **asymptotic growth**, O, Ω, Θ, o and ω notations
- **polynomial and non-polynomial problems,** deterministic and non-deterministic algorithms, P- class problems, NP-class of problems, Polynomial problem reduction NP complete problems- vertex cover and 3-SAT and NP hard problem - Hamiltonian cycle.

TG-sppu_te_be_comp

Analysis

Input Size:

Input size (number of elements in the input)

- **Running time depends on input size**
- Measure time efficiency as function of input size
- Different input of size n may take a different amount of time

TG:sppu_te_be_comp

How do we determine the input size?

1. A natural parameter
2. For sorting and other problems on array: Array size
3. For combinatorial problems: No of objects
4. For graphs: no of vertices and edges

The compile time does not depend on the instance characteristics. But **run time is depending on the instance characteristics.**

f(n) :Where n denotes the instance characteristics

TG:spou_te_be_comp

Types of Analysis

Efficiency of algorithms, is known as analysis of algorithms.

The function f(n), gives the running time of an algorithm, depends not only on the size 'n' of the input data but also on the particular data.

The complexity function f(n) for certain cases are:

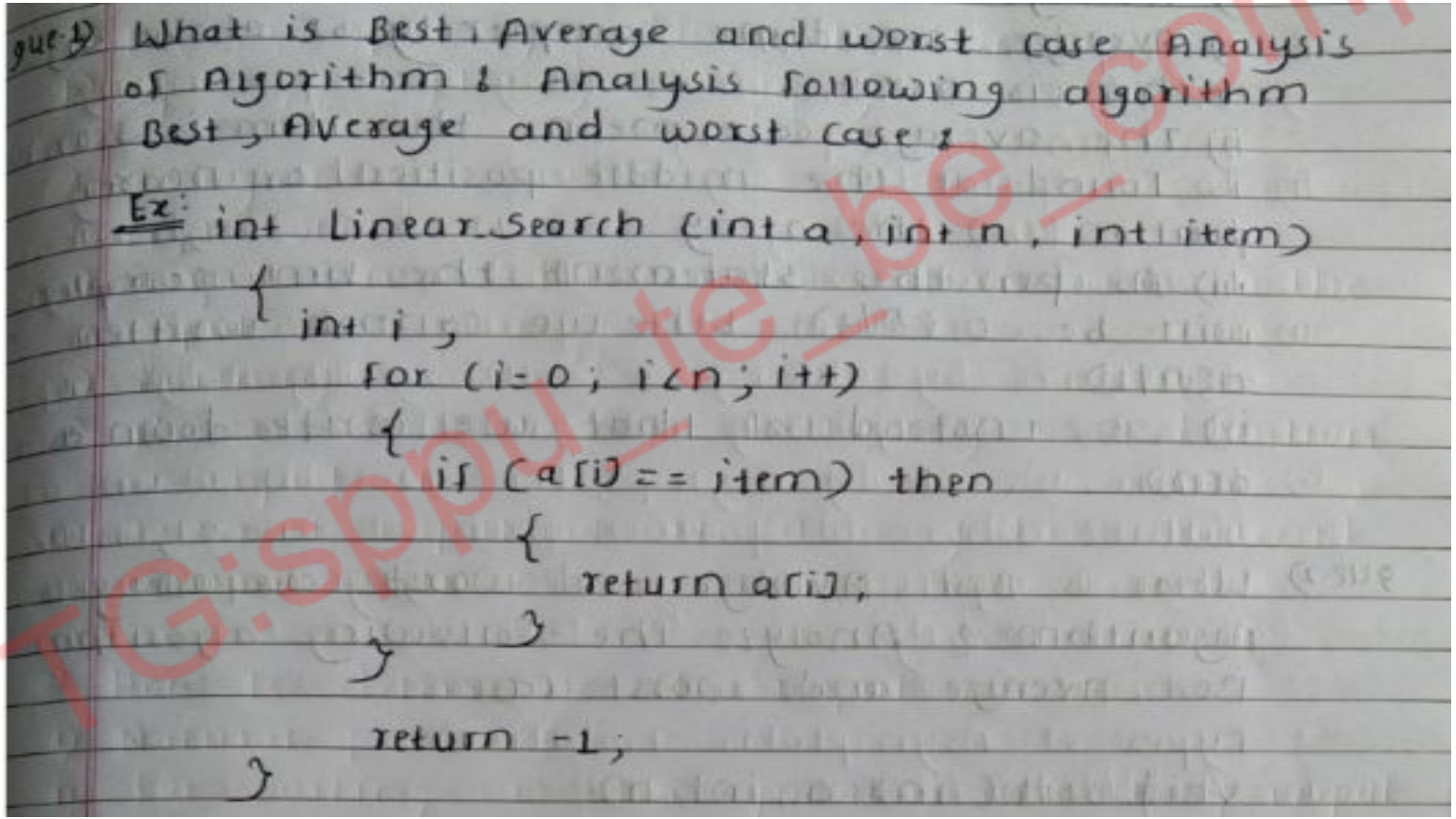
TG:sppu_te_be_comp

Cont...

1. **Best Case :** The minimum possible value of f(n) is called the best case.
   - Provides a lower bound on running time
   - Input is the one for which the algorithm runs the fastest
2. **Worst Case :** The maximum value of f(n) for any key possible input.
   - Provides an upper bound on running time
   - An absolute guarantee that the algorithm would not run longer, no matter what the inputs are
3. **Average Case :** The expected value of f(n).
   - Provides a prediction about the running time
   - Assumes that the input is random

TG:sppu_te_be_comp

Example- Linear Search



TG:sppu_te_be_comp

Example- Linear Search

→ For above algorithm,

a) Best case

Best case is O(1). this is a case when the item to be searched in a array found at first position.

b) Worst case

i) worst case time complexity of above algorithm is O(n)

ii) Here, worst case means the item to be searched in the given array is found at last position or not found at all.

c) Average case

i) Average case time complexity of above algorithm is O(n)

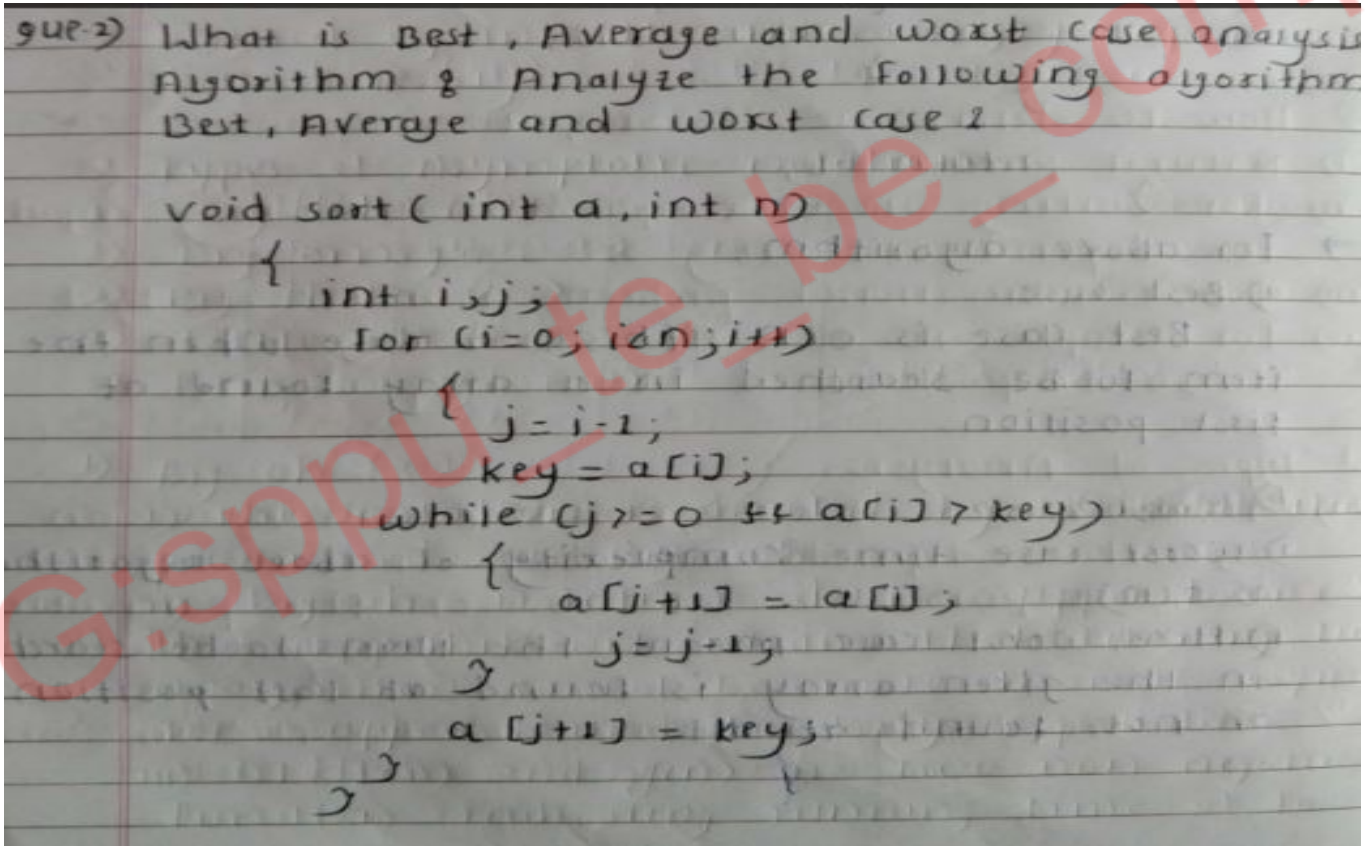
ii) The average case means the item is search is found at the middle position or nearer position of the array

iii) As per this statement the time complexi will be O(n/2) but we cannot written O(n/2)

iv) so, instead of that we write down as a O(n).

TG-sppu_te_comp

Example- Sorting



Que-2) What is Best, Average and Worst case analysis Algorithm & Analyze the following algorithm Best, Average and Worst case ?

```
void sort ( int a, int n)
  {
    int i,j;
      for (i=0; i<n; i++)
        {
          j = i-1;
          key = a[i];
          while (j>=0 && a[i] > key)
            {
              a[j+1] = a[j];
              j=j-1;
            }
          a[j+1] = key;
        }
  }
```

TG:sppu_te_be_comp

Example- Sorting

a) Best Case

i) Best Case time complexity of above algorithm is $O(n)$.

ii) Best Case is when the given array is already sorted, in that case the inner while loop will iterate only one in a while loop

iii) so, time complexity completely depend on For loop and as a for loop is iterating n times

iv) so, the Best Case time complexity here is $O(n)$.

b) Worst Case

i) When we want to sort array in ascending order but given array is in descending order in that case the inner while loop will iterate maximum n time and outer For loop will also iterate till n time.

ii) so, the worst case time complexity will be $n \times n$ i.e $O(n^2)$

c) Average Case

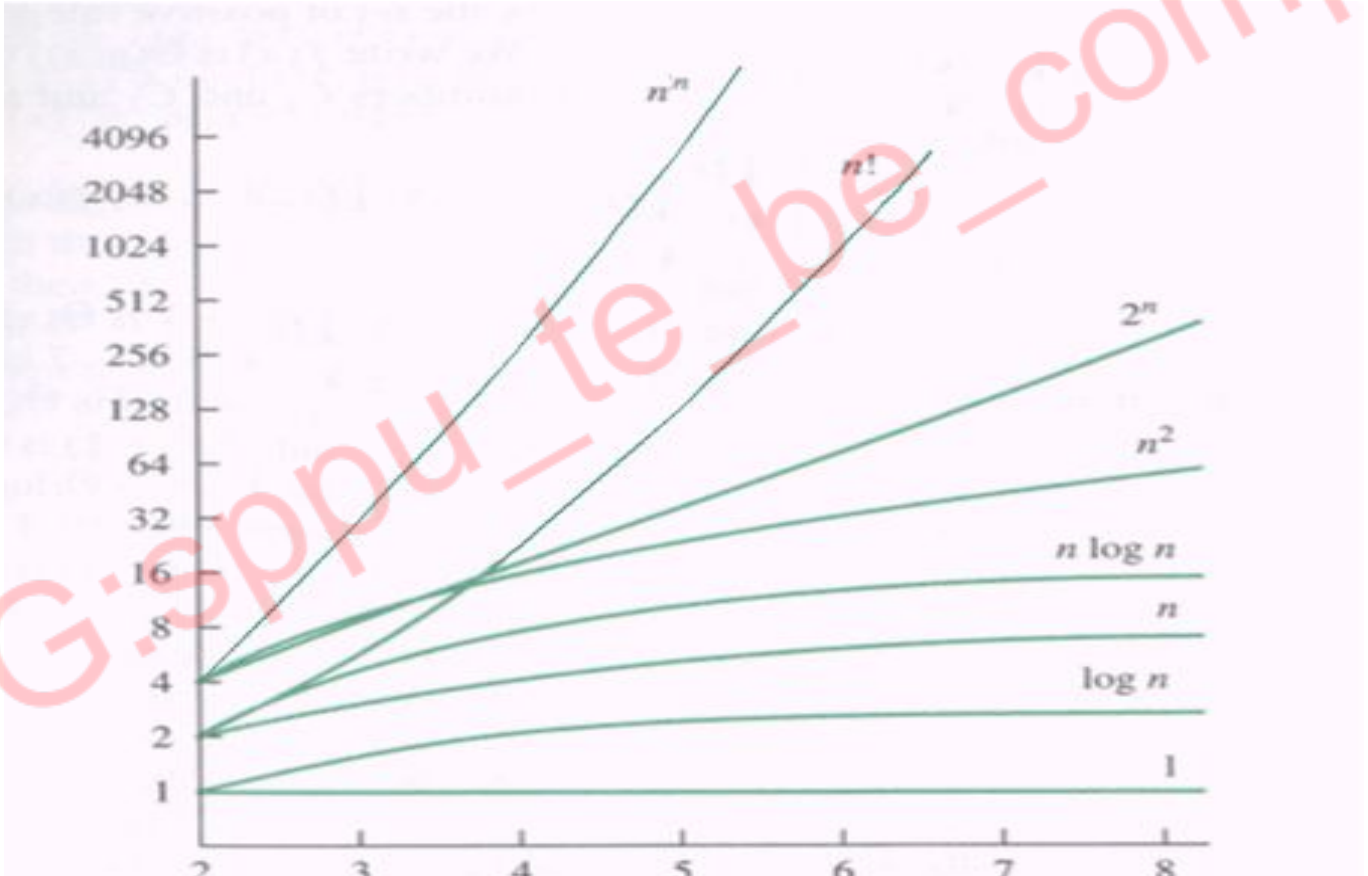
i) In Average Case, the array is having random input.

ii) So, in this case outer loop will iterate n time and inner loop will iterate $n/2$ time

iii) so, the time complexity will be $n \times \frac{n}{2} = \frac{n^2}{2}$ which is equal to $O(n^2)$

TG-sppu_te_be_comp

Rate of Growth



TG:sppu_te_be_comp

Asymptotic Growth

- It is **mathematical way of representing the time complexity.**
- To compare two algorithms with running times ***f(n)*** **and** ***g(n)***, we need a **rough measure** that characterizes **how fast each function grows. [Use rate of growth]**
- Compare functions in the limit, that is, **asymptotically!**

(i.e., for large values of *n*)

TG:sppu_to_be_comp

Asymptotic Growth

- O notation: asymptotic "less than":
  - f(n)=O(g(n)) implies: f(n) "≤" g(n)
- Ω notation: asymptotic "greater than":
  - f(n)= Ω (g(n)) implies: f(n) "≥" g(n)
- Θ notation: asymptotic "equality":
  - f(n)= Θ (g(n)) implies: f(n) "=" g(n)

TC·sppu_fe_be_comp

Big-O Notation

- We say $f_A(n) = 30n + 8$ is order n, or O (n)

  It is, at most, roughly proportional to n.

- $f_B(n) = n^2 + 1$ is order n^2, or $O(n^2)$.

- It is, at most, roughly proportional to n^2.

- In general, any $O(n^2)$ function is faster- growing than any O(n) function.

TG:sppu_te_be_comp

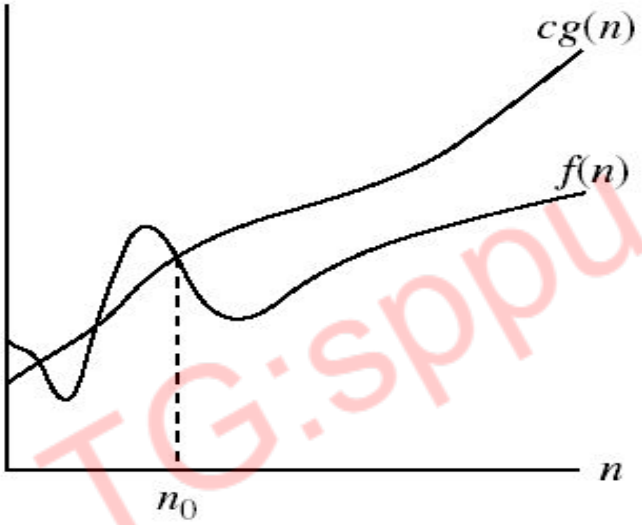
Examples

- $n^4 + 100n^2 + 10n + 50$ is $O(n^4)$
- $10n^3 + 2n^2$ is $O(n^3)$
- $n^3 - n^2$ is $O(n^3)$
- constants
  - 10 is O(1)
  - 1273 is O(1)

TG:sppu_te_be_comp

Big Oh Notation

$O(g(n)) = \{f(n) :$ there exist positive constants c and n_0 such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0\}$.



$g(n)$ is an ***asymptotic upper bound*** for $f(n)$.

TG:sppu_te_be_comp

More Examples

- f(n)=Og(n)

f(n)<=c*g(n)

Let f(n)=2n+3

- 1. 2n+3<=10n ; True ,n>=1 : f(n)=O(n)
- 2. 2n+3<=2n+3n=5n ; True, n>=1 : f(n)=O(n)
- 3. 2n+3<=2n^2+3n^2=5n ^2 ; True, n>=1 : f(n)=O(n^2)

If g(n)=log n then , f(n)=O(log n); not true

So closest func for Big O is f(n)=O(n)

TG:sppu_te_be_comp

gue.1) Find upper bound of running time of Linear funcion $f(n) = 6n+3$

→ soln: We have to find value of c & n_0 such that,

$= 0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

$= 0 \leq 6n+3 \leq 6n+3n$

$= 0 \leq 6n+3 \leq 9n$

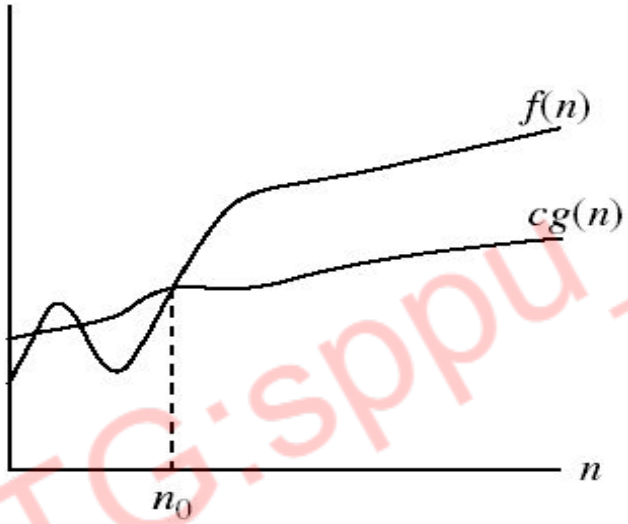
i/p n	f(n) 6n+3	c·g(n) 9n	
1	9	9	same value so, $n_0 = 1$
2	15	18	
3	21	27	
4	27	36	

From above table we can say that $f(n) = O \cdot g(n)$ for $c = 9$ and $n_0 = 1$

TG-sppu_te_be_comp

Ω - notation

$\Omega(g(n)) = \{f(n)$: there exist positive constants c and n_0 such that $0 \leq cg(n) \leq f(n)$ for all $n \geq n_0\}$.



$g(n)$ is an ***asymptotic lower bound*** for $f(n)$.

TG:sppu_te_be_comp

Examples

- f(n)= Ω g(n)

f(n)>=c*g(n)

Let f(n)=2n+3

- 1. 2n+3>=2n ; True ,n>=1 : f(n)= Ω(n)
- 2. 2n+3>=log n ; True, n>=1 : f(n)= Ω(log n)

If g(n)= n^2 then , f(n)= Ω(n^2); not true

So closest func for Big Ω is f(n)= Ω(n)

TG.sppu_te_be_comp

Que.1) Find lower bound of running time of linear function $f(n) = 6n+3$

→ Soln: We have to find c and n_0 such that

$0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$

$= 0 \leq c \cdot g(n) \leq 6n+3$

$= 0 \leq 6n \leq 6n+3$

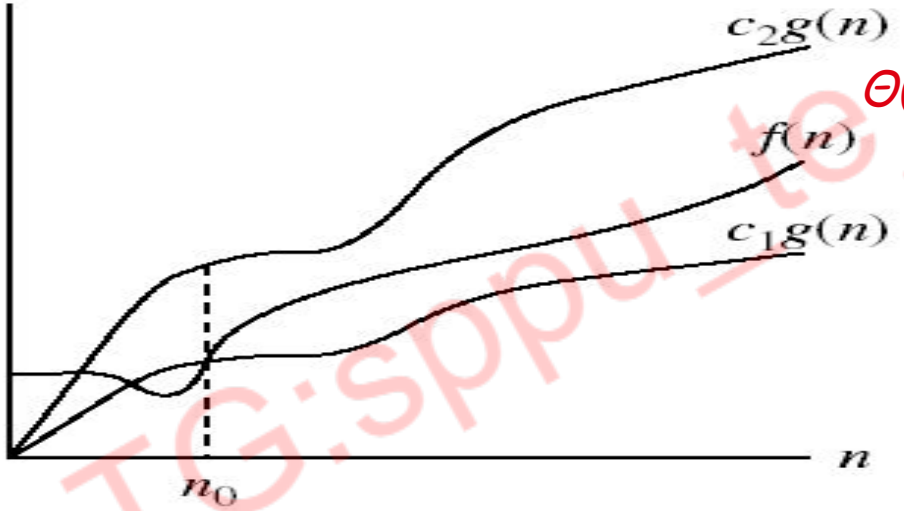
i/p n	f(n) $6n+3$	c g(n) $6n$	
1	9	6	$\therefore 9 > 6$
2	15	12	
3	21	18	

From above table we can say that $f(n) = \Omega(n)$ for $c = 6$ and $n_0 = 1$.

TG:sppu_te_be_comp

Θ-notation

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2, \text{ and } n_0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0\} .$$



Θ(g(n)) is the set of functions with the same order of growth as g(n)

$g(n)$ is an ***asymptotically tight bound*** for $f(n)$.

TG:sppu_to_be_comp

Examples

- f(n)= Θ g(n)

C1*g(n)<=f(n)<=c2*g(n)

Let f(n)=2n+3

1*n<= 2n+3<=5n ; True ,n>=1 : f(n)= Θ(n)

If g(n)= n^2 then , f(n)= Θ(n^2); not true

If g(n)= log n then , f(n)= Θ(log n); not true

So func is exact equal to f(n)= Θ(n)

TG:sppu_te_be_comp

Little o

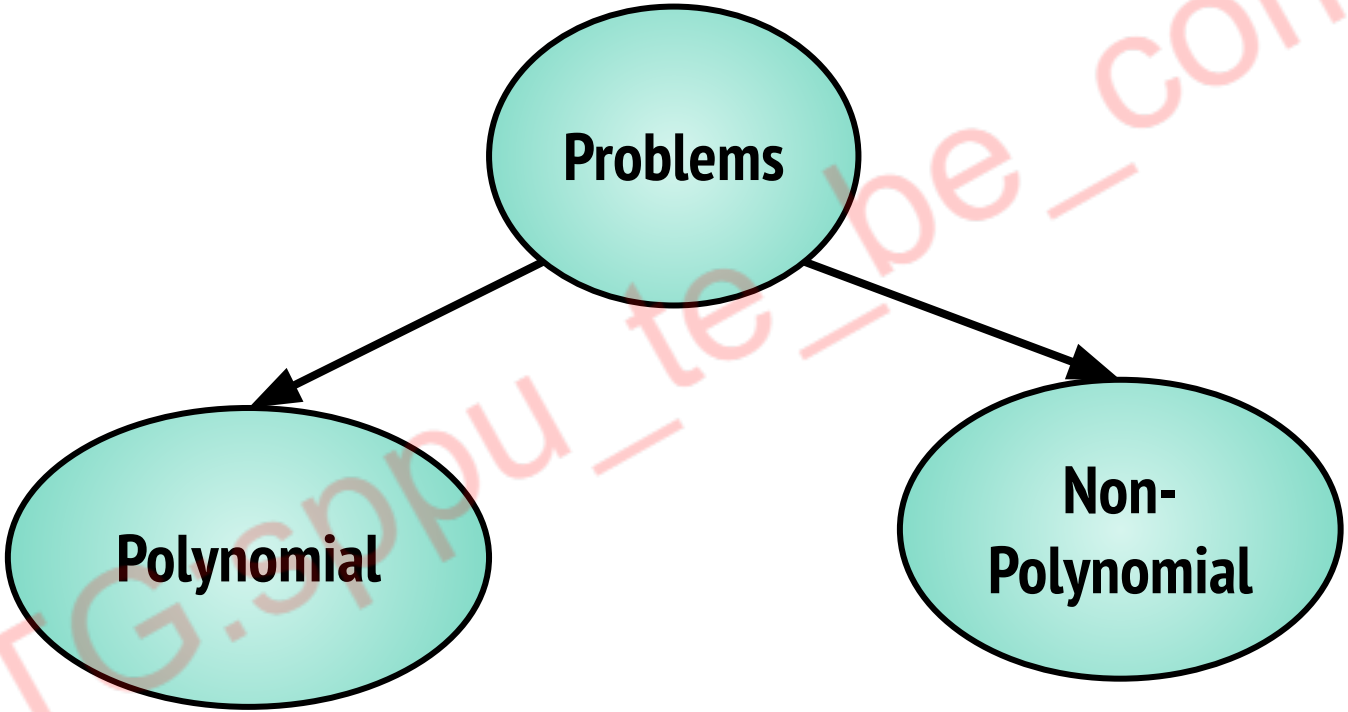
- We use o-notation to denote an upper bound that is not asymptotically tight
- We formally define o.g.n// (“little-oh of g of n”) as the set
- o.g.n// D ff .n/ W for any positive constant c>0, there exists a constant n0 > 0 such that 0 f .n/ < cg.n/ for all n n0g :

TG-Spot_te_be_comp

Complexity Theory

TG:sppu_te_be_comp

Types of Problem



TG:sppu_te_be_comp

Types of Problem

- **Polynomial- Problems that can be solved by a polynomial time algorithm**
  - **Example-**
  - Searching o(logn)
  - Sorting o(nlogn)
  - String Editing o(mn)
- **Non Polynomial- Problems for which no polynomial time algorithm is known**
  - **Example-**
  - Travelling Salesperson $O(n^2 2^n)$
  - Knapsack Problem $O(2^{n/2})$

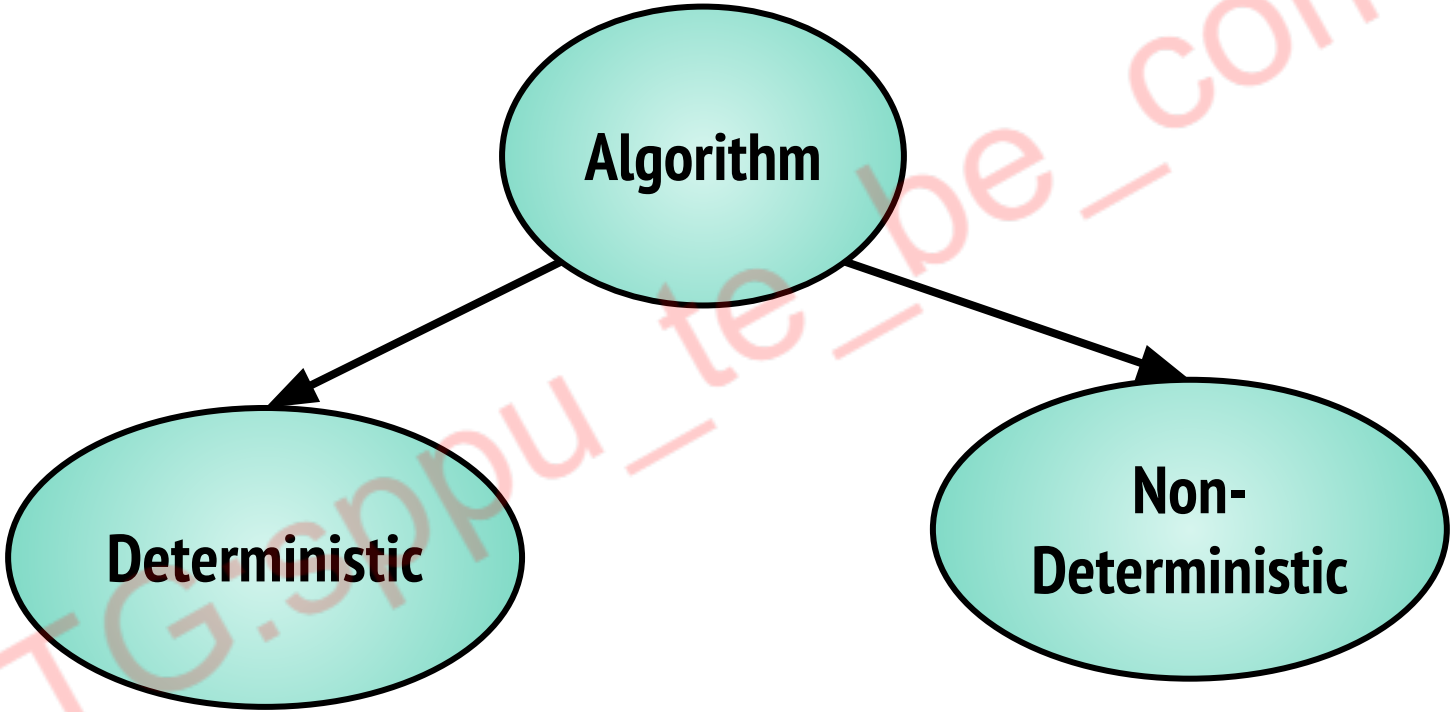
TG-sppu_te_be_comp

Types of Problem

constant	$O(1)$
logarithmic	$O(\log n)$
linear	$O(n)$
n-log-n	$O(n \times \log n)$
quadratic	$O(n^2)$
cubic	$O(n^3)$
exponential	$O(k^n)$, e.g. $O(2^n)$
factorial	$O(n!)$
super-exponential	e.g. $O(n^n)$

TG:shou_te_ke_comp

Types of Algorithm



TG.sppu_te_be_comp

Types of Algorithm

- **Deterministic Algorithm-**
  - Result of every operation is uniquely defined
  - For a particular input computer will give always same output
  - Can solve problem in Polynomial time
  - Can determine the next step of execution
  - **Example-**
  - 5 + 3 =8
- **Non Deterministic Algorithm-**
  - **For same input, Compiler may produce different output in different runs.**
  - Can not solve problem in Polynomial time
  - Can not determine the next step of execution due to more than one path the algorithm ca take
  - **Example-**
  - X + Y =21 (Which 2 numbers sum equal to 21)

TGsppu_te_be_comp

Non Deterministic Algorithm

NDC Algorithm:

some of the terms related to the non-deterministic algorithm are defined below:

1. **choice(X) :** chooses any value randomly from the set X.
2. **failure() :** denotes the unsuccessful solution.
3. **success() :** Solution is successful and current thread terminates.

TG:sppu_te_be_comp

Consider the problem of searching for an element x in a given set of elements A[1:n]

Non Deterministic search

```
j= choice(a, n)
if(A[j]==x) then
{ write(j); success(); }
write(0); failure();
```

TG:sppu_te_be_comp

Decision Problem Vs Optimization Problems

Decision Problem	Optimization Problem
Any problem for w**hich the answer is either Zero or One (Yes or No)** is called a decision problem	Any problem that involves the **identification of an optimal (either maximum or minimum)** value of a given function is known as an optimization problem
An algorithm for a decision problem is terms as decision problem	An optimization algorithm is used to solve an optimization problem
Deterministic problems are easier as compared to Optimization problem	Optimization problems are difficult as compared to Deterministic problem

TG:sppu_te_be_comp

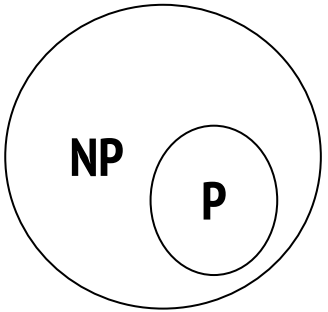
P, NP, NP Hard, NP Complete

Computational Complexity Problems

TG:sppu_te_be_comp

P, NP

- **P-** Problems that can be solved in **Polynomial time** (“P Stands for Polynomial”) using **deterministic algorithm**
  - Eg. Searching, Sorting etc.
- **NP**- It stands for **non- deterministic polynomial time.**
  - It is a **collection of decision problems that can be solved by a non-deterministic machine in polynomial time.**
  - Since deterministic algorithm are just a special case of non-deterministic once,
  - we conclude that P is Subset of NP
  - NP Class problems can be further categorized into NP-Complete & NP-Hard Problems



TC-sppu_te_be_comp

P Vs NP

P	NP
P Problems are set of **problems which can be solved in polynomial time by deterministic algorithm**	NP Problems are **set of problems which can be solved in polynomial time by Non-deterministic algorithm**
P problems can be **solved & Verified in Polynomial Time**	Solution to NP problems **cannot be Solved/Obtained in Polynomial time, but if the solution is given, it can be verified in Polynomial time.**
P Problems are subset of NP Problems	NP Problems are Superset of P Problems
All P Problems are deterministic in Nature	All NP Problems are non-deterministic in nature
Eg. Selection Sort, Linear Search	Eg. TSP, Knapsack Problem

TG:Sppu_te_be_comp

NP Complete

- NP Complete- A problem is called NP- Complete if

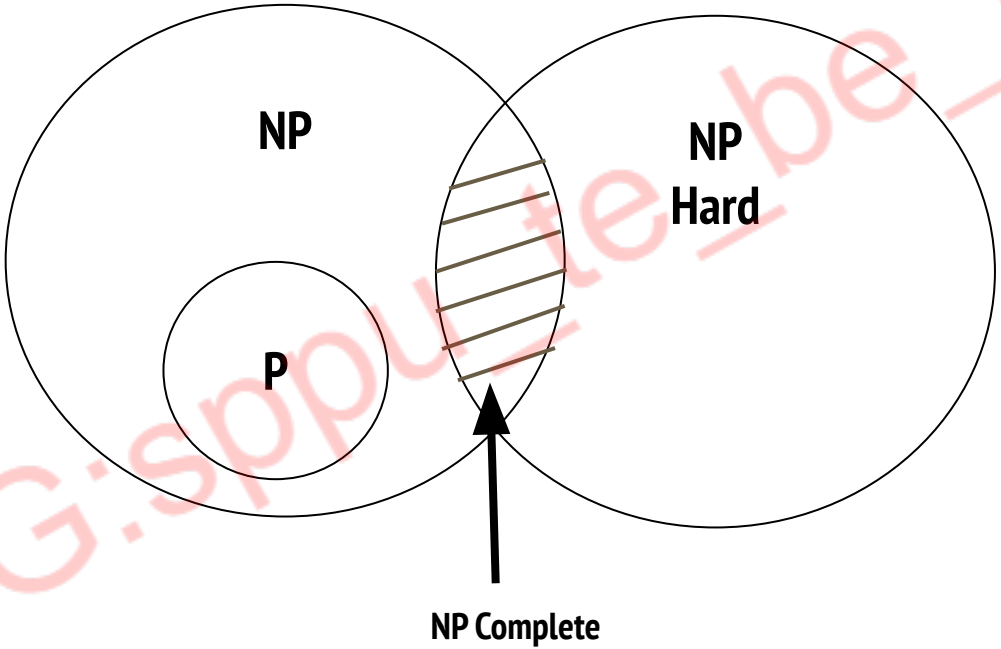
    1. **It belongs to class NP**

    2. **Every other problems B in NP can be transformed to A in Polynomial time.**

- These 2 facts prove that NP- Complete problems are the hardest problems in Class NP.
- However a solution of any NP-Complete Problem can be verified in Polynomial time but cannot be obtained in Polynomial time.

TG:sppu_te_be_comp

Relation Between P, NP, NP Hard, NP Complete



TG:sppute_be_comp

NP-Hard Vs NP-Complete

NP Hard	NP Complete
NP Hard problem(Say A) can be solved if & only if there is NP Complete problem (Say B) can be reducible into A in Polynomial time.	NP Complete problems can be solved by deterministic algorithm in Polynomial time
To solve NP Hard problem, it must be in NP class	To solve NP Complete problems, it must be in both NP and NP hard problems
It is not a decision problems	It is a decision Problems
Eg. Halting Problems, Hamiltonian Cycle Problems	Eg. Determine whether a graph has a hamiltonian cycle or not.

TG:sppu_te_be_comp

Polynomial Problem Reduction

- Let L1 and L2 be problems
- We can say that Problem L1 reduces to L2, if we have a Polynomial time algorithm for L2, then we can also solve L1 in Polynomial time.
- If Problem P1 is NP-Complete & there is polynomial time reduction of P1 to P2 then P2 is also NP-Complete.

TG:sppu_te_be_comp

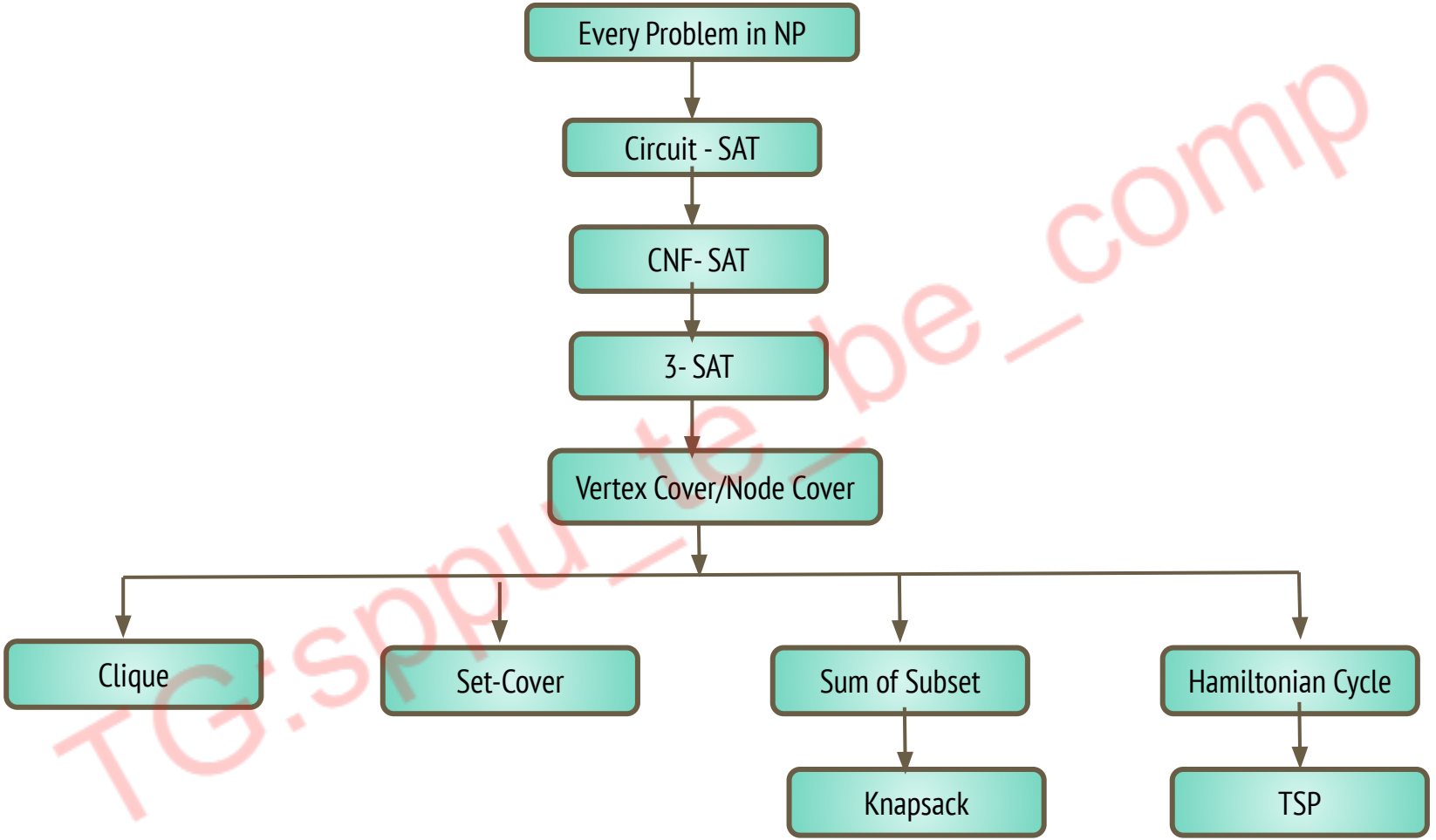


Fig- Reduction in NP- Completeness

TG:sppu_te_be_comp

Steps to Prove Problem L2 is NP- Complete

1. To show that Problem L2 is NP Hard
2. An NP-Hard decision Problem L2 can be shown to be NP -Complete by exhibiting a Polynomial time non- deterministic algorithm for L2

Steps to Prove Problem L2 is NP- Hard

1. Pick a Problem L1 already known to be NP-Hard
2. Show how to obtain(in Polynomial time) an instance I' of L2 from any instance I of L1, such that from solution of I' we can determine the solution to I (In polynomial deterministic time)
3. Conclude from step 2 that L1 is Reducible to L1
4. Conclude from Step 31 and 3 that Problem L2 is also NP Hard.

TG:spu_te_be_comp

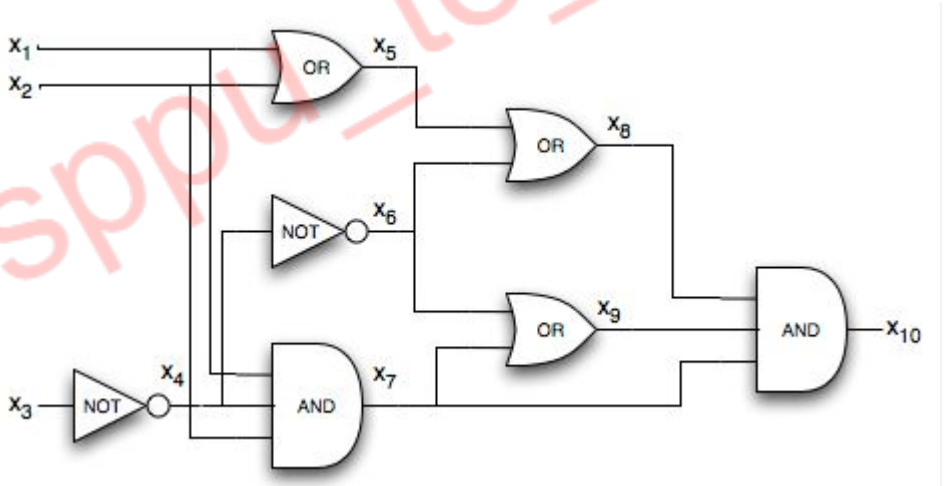
SAT/Satisfiability Means?

- SAT means Satisfiability
- What is meant by Satisfiability?-
  - Let X1, X2…Xn denote boolean variables.
  - $\overline{X_i}$ Denote the negation of Xi. A literal is either a variable or its negation
  - A Formula is the propositional calculus is an expression that can be constructed using literals and with operations and, or
  - Symbols $\vee$ denotes or $\wedge$ denotes and
  - CNF is Formula like $(X_1 \vee X_2) \wedge (X_2 \vee X_3)$
  - Satisfiability problem is to determine whether a formula is true for some assignment of truth values to a variables

TG:sppu_te_be_comp

Circuit SAT

- This is a problem in which a Boolean circuit is taken as input with single output node.
- And then finds whether there is an assignment of values to the circuit's input so that we get its output values as "1".
- This assignment of values is called satisfying assignment



TG:sppu_te_be_comp

Theorem: Circuit SAT is in NP

- Lets Construct a non-deterministic algorithm, which works in polynomial time.
- In which there is choose() method which can guess the values of inputs and output.
- If answer if 0 output will be No
- If answer is 1 output will be Yes
- If algorithm has output output as “Yes” then we can say that Boolean circuit has satisfying input values.
- Thus we can say that Circuit-SAT is in NP.

TG.sppu_te_be_comp

What is 3-SAT Problem (Restricted Satisfiability)?

- 3-SAT Problem is the problem of determining the satisfiability of a CNF formula where each clause is limited to at most 3 literals
- Example:

$(X_1 \vee X_2 \vee X_3) \wedge (X_2 \vee X_3 \vee X_4)$

TG:sppu_te_be_comp

Prove that 3-SAT is NP Complete

Proof:

1. 3-SAt is NP. Given an assignment we can just check that each clause is covered.
2. 3-SAT is NP- hard,
   To Prove this, a reduction from SAT to 3-SAT must be provided.
3. We will transform each clause independently based on its length.

TG:sppu_te_be_comp

Prove that 3-SAT is NP Complete

iv) Reduction from SAT to 3-SAT

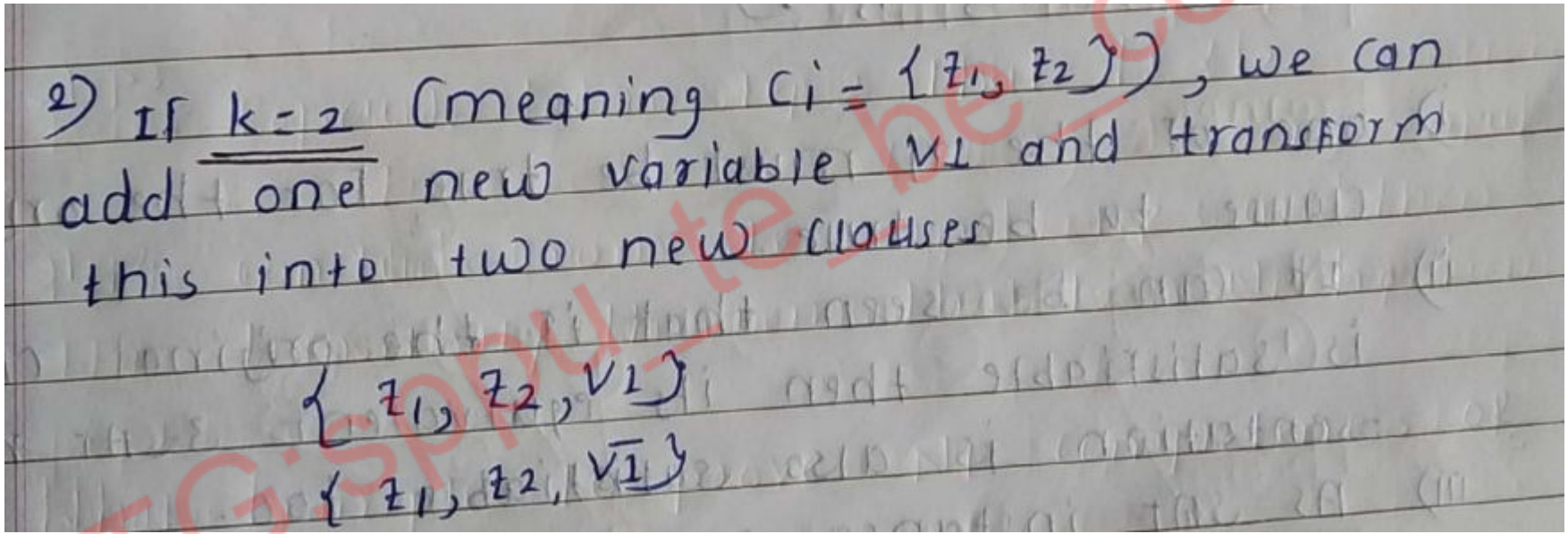
Suppose a clause contains k-literals

1) If k=1 (meaning $C_i = \{z_1\}$), we can add two new variables v_1 & v_2 and transform this into four clauses

$(z_1 \vee v_1 \vee v_2) \wedge (z_1 \vee \bar{v_1} \vee \bar{v_2}) \wedge (z_1 \vee v_1 \vee \bar{v_2}) \wedge (z_1 \vee \bar{v_1} \vee v_2)$

TG_sppu_te_be_comp

Prove that 3-SAT is NP Complete



TG-spru_te_be_comp

Prove that 3-SAT is NP Complete

3) If $k = 3$ (meaning $c_i = \{z_1, z_2, z_3\}$), we move the clause as it is, no need to add any new variable.

4) If $k > 3$ (means $c_i = \{z_1, z_2, z_3, \ldots, z_k\}$), we can add in $k-3$ new variables $\{v_1, v_2, \ldots, v_{k-3}\}$ and $k-2$ clauses.

$\{z_1, z_2, v_1\}$

TE-sppu-te_be_comp

Prove that 3-SAT is NP Complete

Example:

if $k = 4$ then $k-3$ means $4-3=1$ new variables we have to introduced.

So, new variable will be v_1 and $\overline{v_1}$

And $k-2$ means $4-2=2$ clauses will be created.

given Four variables are z_1, z_2, z_3, z_4 and new variables v_1 and $\overline{v_1}$.

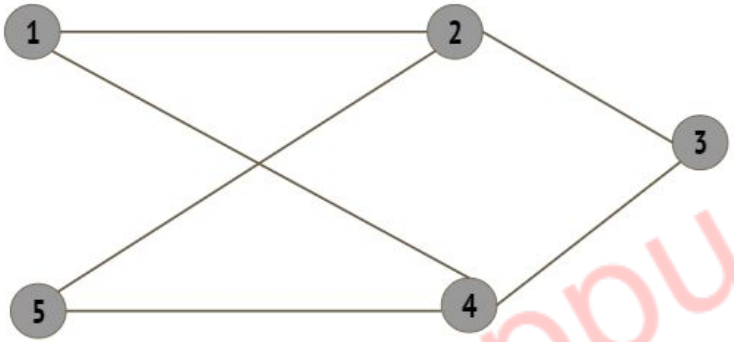
two clauses that will be created,

$\{z_1, z_2, v_1\}$

$\{z_3, z_4, \overline{v_1}\}$

TG:sppu_te_be_comp

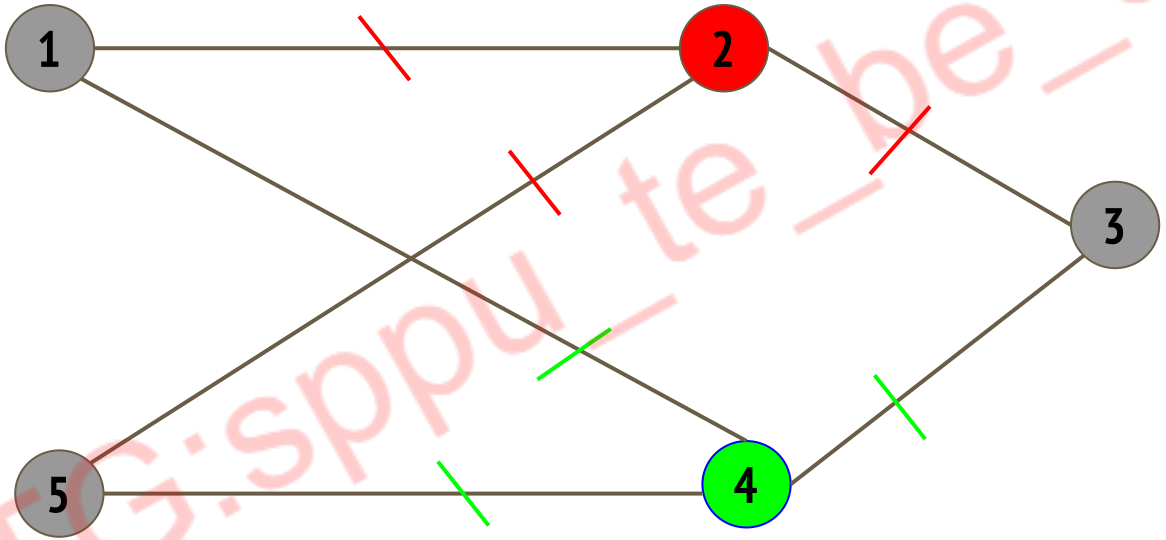
Node Cover/Vertex Cover



- Node Cover Problem is to find node cover of minimum size in a given graph.
- The word node cover means each node covers its incident edges.
- Thus by node cover we expect to choose the set of vertices which cover all edges in the graph.

TG:sppu_te_be_comp

Node Cover/Vertex Cover



- Node Cover for this example is {2,4} of size 2

TG:sppu_te_be_comp

Theorem- Node Cover is NP Complete

Proof: To Prove this we require to follow steps given below

1. Take a problem which known to be NP Hard, and if that problem is reducible to Node cover then Node Cover is also NP-Hard
2. Proof that Node Cover is NP
3. If Node is NP and NP-Hard we can say that Node Cover is NP Complete

TG:sppu_te_be_comp

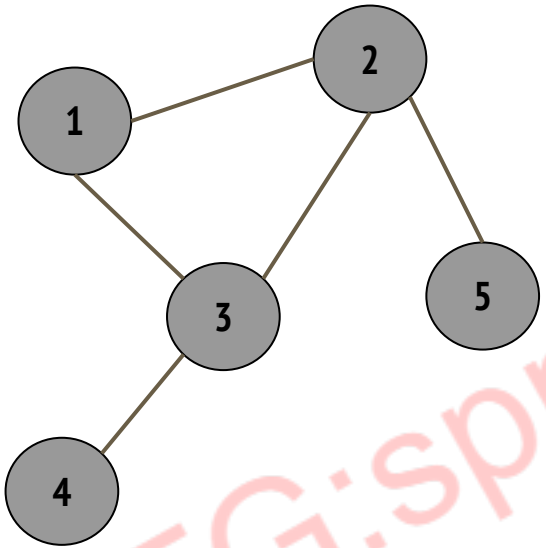
Theorem- Node Cover is NP Complete

Proof: To Prove this we require to follow steps given below

1. Take a problem CDP (Clique Decision Problem) which known to be NP Hard, and we try to reduce this problem instance into instance of Node cover.

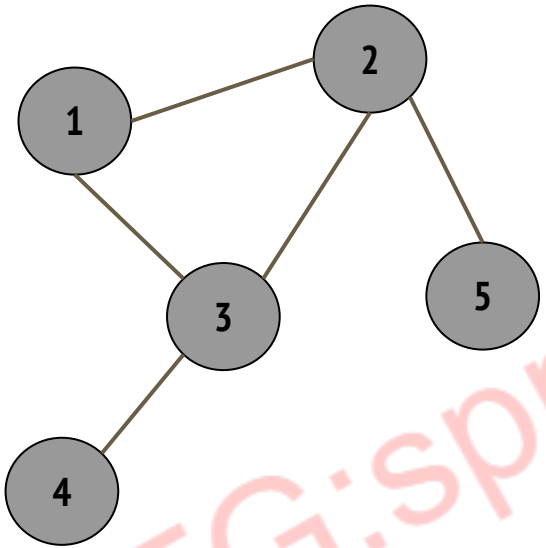
TG:sppu_te_be_comp

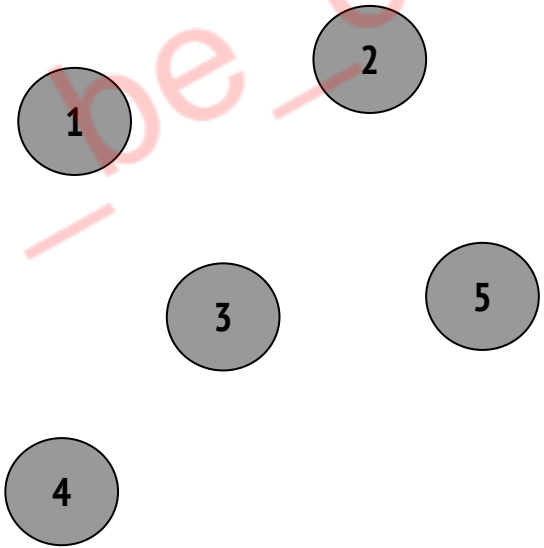
Theorem- Node Cover is NP Complete



TG:sppu_te_be_comp

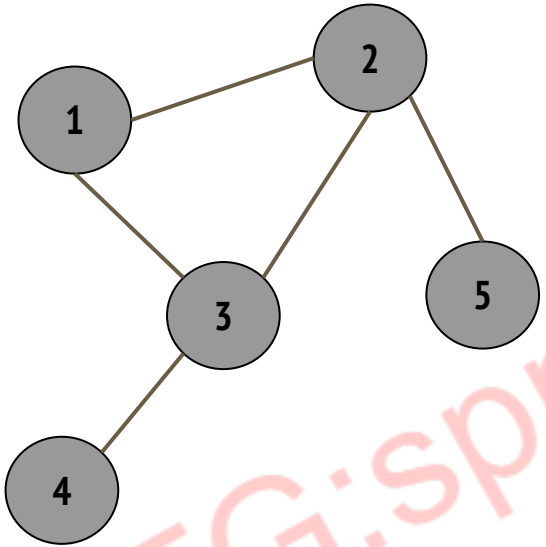
Theorem- Node Cover is NP Complete

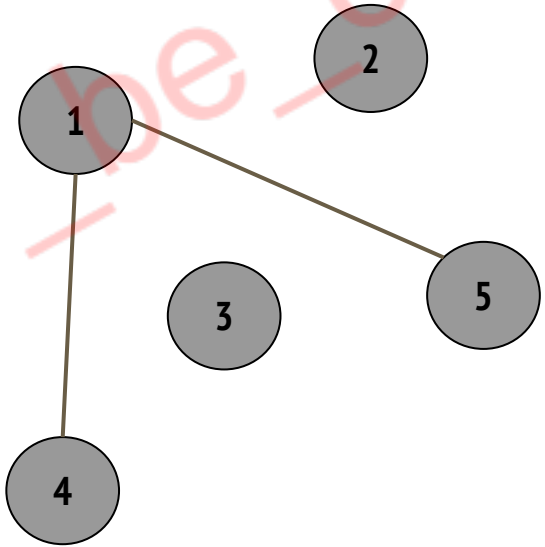




TG:sppu_te_be_comp

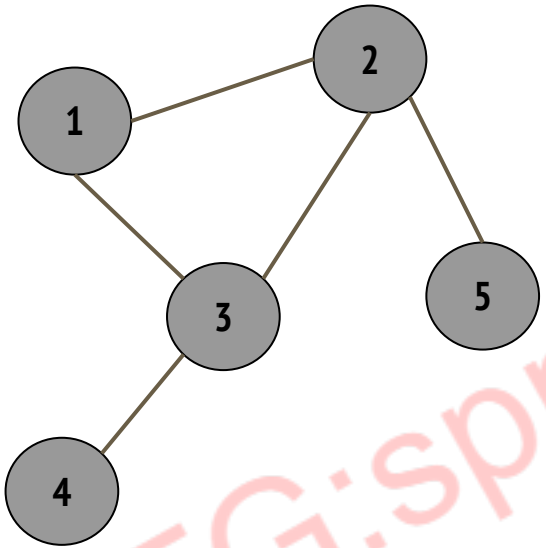
Theorem- Node Cover is NP Complete

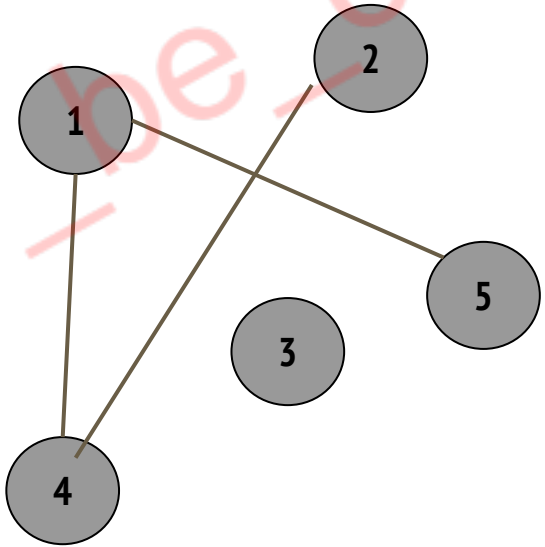




TG:sppu_te_be_comp

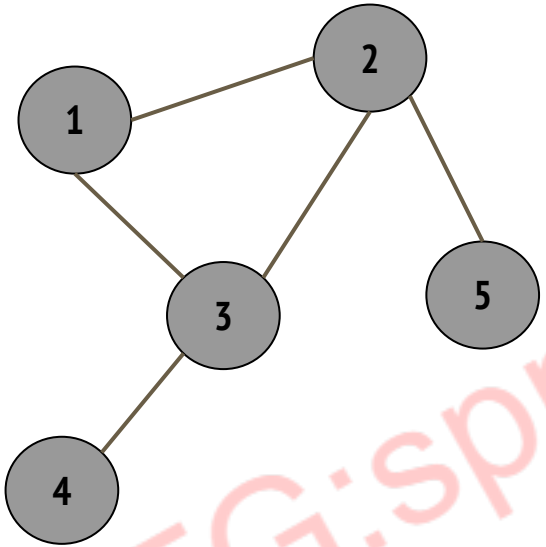
Theorem- Node Cover is NP Complete

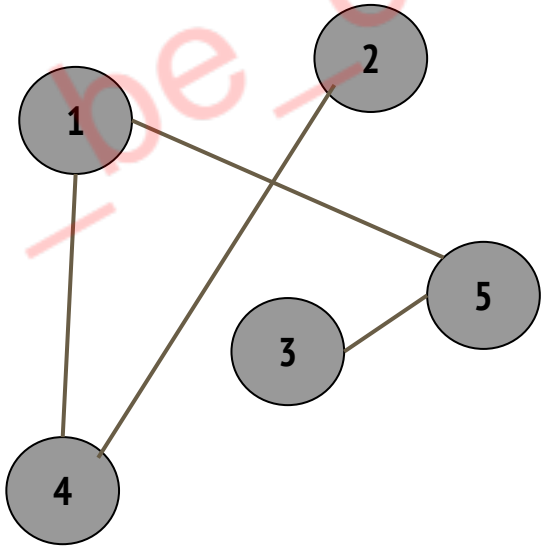




TG:sppu_te_be_comp

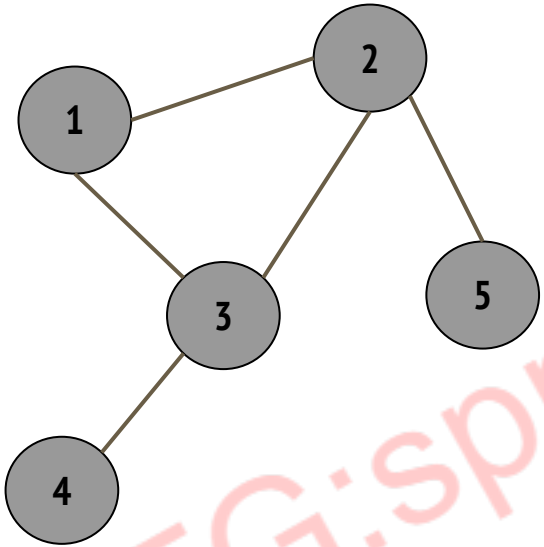
Theorem- Node Cover is NP Complete



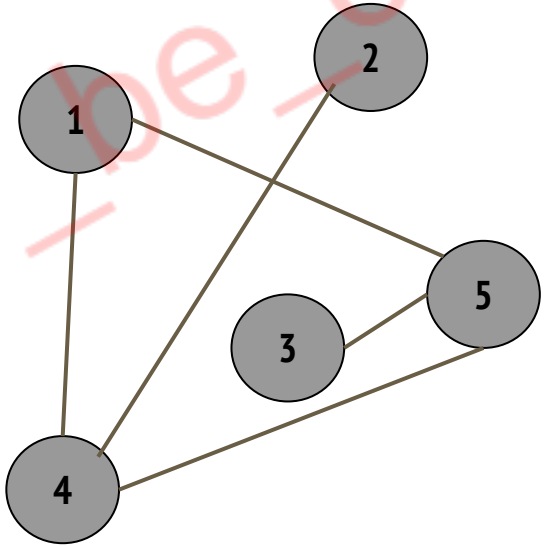


TG:sppu_te_be_comp

Theorem- Node Cover is NP Complete



Graph- G
Clique of Size 3 with Vertex {1,2,3}



Graph- G'
Node Cover of Size 2 with Vertex {4,5}

TG:sppu_te_be_comp